國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab08

姓名：李品融

學號：113310217

Lab08-Q1Q2
這兩題在練習二元樹的遍歷轉換與子樹極值搜尋。先將二元樹的中序遍歷修改為後序遍歷，接著再實作遞迴演算法，找出根節點左、右兩完整子樹的最大數值。

Code

```
#include <iostream>
#include <queue>
#include <vector>
#include <algorithm> // 用於 std::max 函數來比較大小
#include <climits>   // 用於 INT_MIN (代表整數的最小值)

using namespace std;

// 樹的節點
class TreeNode {
public:
 int value;            // 節點的值
 TreeNode* left;       // 左子節點
 TreeNode* right;      // 右子節點
 TreeNode(int val) : value(val), left(NULL), right(NULL) {}  // 初始化節點，將左右節點指標設為空
};

// 樹結構
class BinaryTree {
public:
 TreeNode* root;
 BinaryTree() : root(NULL) {} // 初始化樹，跟節點指向 NULL

 // 將陣列轉換為二元樹
 void buildTree(vector<int>& arr) {
  if (arr.empty()) return; // 如果陣列為空，直接返回

  queue<TreeNode*> q; // 暫存已經建立但還沒分配子節點的父節點
  root = new TreeNode(arr[0]); // 陣列的第一個元素作為根節點
  q.push(root); // 將根節點放入隊列

  size_t i = 1; // 陣列索引從 1 開始 (陣列 0 為根節點)
```

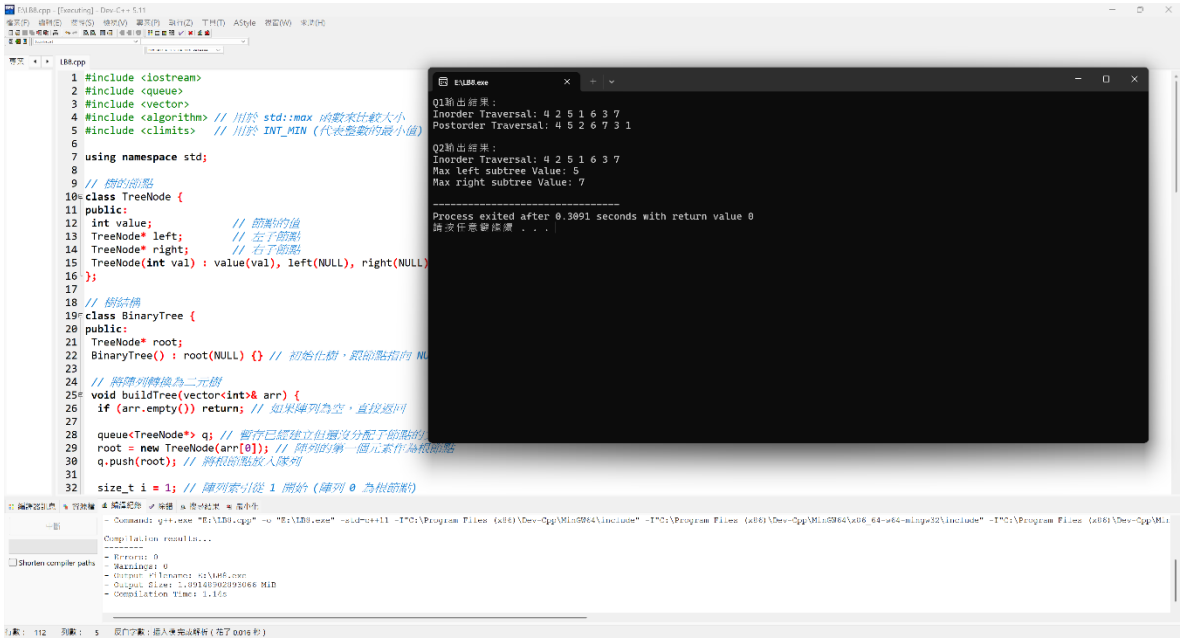
```
 while (!q.empty() && i<arr.size()) {
  TreeNode* current = q.front(); // 隊列最前面取出一個節點當作當前父節點
  q.pop(); // 取出後從隊列移除

  // 添加左子節點
  if (i < arr.size()) {
   current->left = new TreeNode(arr[i]); // 建立新節點放在左邊
   q.push(current->left); // 左子節點加入 queue
   i++;
  }
  // 添加右子節點
  if (i < arr.size()) {
   current->right = new TreeNode(arr[i]); // 建立新節點放在右邊
   q.push(current->right); // 將右子節點加入 queue
   i++;
  }
 }
}

// 中序遍歷
// 左子樹 -> 根節點 -> 右子樹
void inorderTraversal(TreeNode* node) {
 if (node == NULL) return; // 若走到空節點，則遞迴終止並回傳

 inorderTraversal(node->left);  // 往左走到底
 cout << node->value << " ";    // 訪問當前節點數值
 inorderTraversal(node->right); // 往右走到底
}

// Q1:後序遍歷 (Left -> Right -> Root)
// 左子樹 -> 右子樹 -> 根節點
void postorderTraversal(TreeNode* node) {
 if (node == NULL) return; // 若走到空節點，則遞迴終止並回傳
 postorderTraversal(node->left); // 先處理左節點
 postorderTraversal(node->right); // 接著處理右節點
 cout << node->value << " "; // 最後列印出自己 (根)
}
```

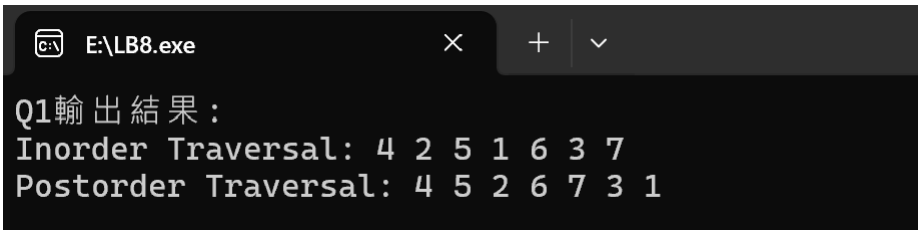
```
 // Q2: 尋找給定子樹最大值
 // 透過遞迴比較自己、左子樹最大值、右子樹最大值
 int findMax(TreeNode* node) {
  if (node == NULL) return INT_MIN; // 如果節點是空的，回傳 INT_MIN

  int res = node->value; // 設當前節點為最大值
  int leftRes = findMax(node->left); // 遞迴找左邊的最大值
  int rightRes = findMax(node->right); // 遞迴找右邊的最大值

  return max({res, leftRes, rightRes}); // 使用 max 函數找出最大值
 }
};

int main() {
 BinaryTree tree; // 宣告二元樹

 // 輸入陣列用於構建樹，NULL 表示空子節點
 vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

 tree.buildTree(arr); // 建立樹

 // Q1 輸出結果:
 cout << "Q1 輸出結果:" << endl;
 cout << "Inorder Traversal: ";
 tree.inorderTraversal(tree.root);
 cout << endl;

 cout << "Postorder Traversal: ";
 tree.postorderTraversal(tree.root);
 cout << "\n" << endl;

 // Q2 輸出結果:
 cout << "Q2 輸出結果:" << endl;
 cout << "Inorder Traversal: ";
 tree.inorderTraversal(tree.root);
 cout << endl;
```

```
 int leftMax = tree.findMax(tree.root->left); // 找出根節點左子樹最大值
 int rightMax = tree.findMax(tree.root->right); // 找出根節點右子樹最大值
 cout << "Max left subtree Value: " << (leftMax == INT_MIN ? 0 : leftMax) <<
endl;
 cout << "Max right subtree Value: " << (rightMax == INT_MIN ? 0 : rightMax) <<
endl;

 return 0;
}
```

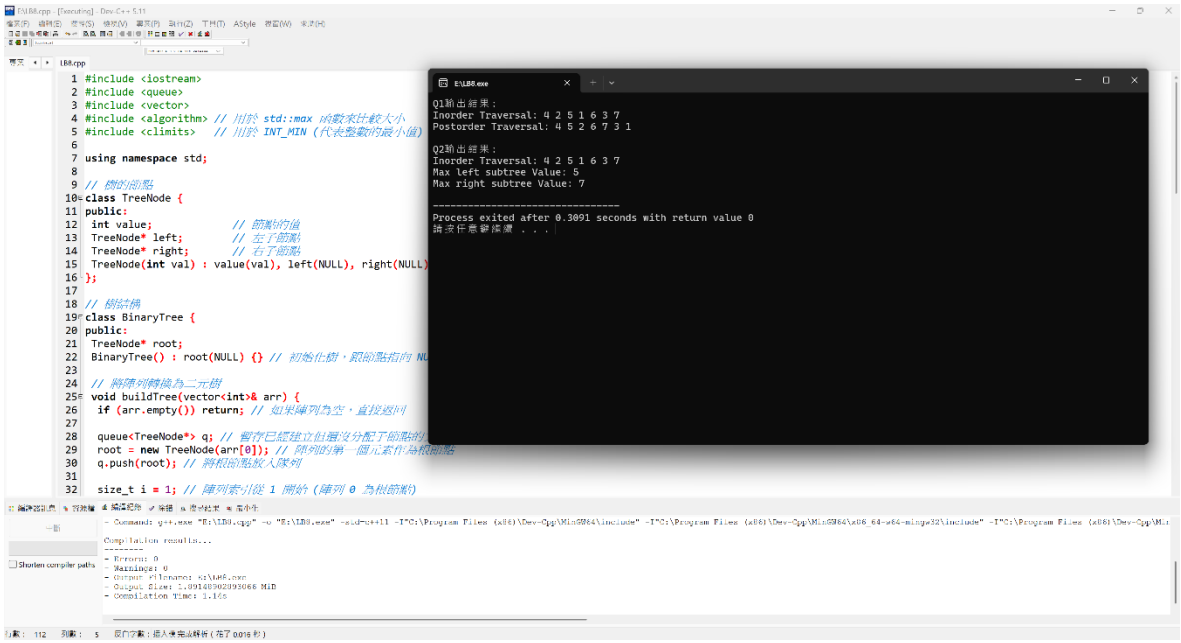
Result

Q1 結果:





Q2 結果:




```
Q2輸出結果:
Inorder Traversal: 4 2 5 1 6 3 7
Max left subtree Value: 5
Max right subtree Value: 7
```

Lab08-Q3
這題在討論，要如何透過演算法來找到左子樹和右子樹中的最大值。
Discussion
首先先透過 Queue 隊實現層序構建，使輸入陣列可以正確對應樹狀階層。接著使用遞迴程式，拆解並比較當前節點與左右子樹的結果，透過 findMax 函數遍歷指定子樹的所有節點，並利用 INT_MIN 常數處理邊界條件，最後精確定位左右子數中的最大值。

Lab08-Q4
這題在討論，若左、右子樹不是完全二元樹，會發生什麼事情?
Discussion
若左、右子樹不是完全二元樹，則在建構樹的階段，當輸入陣列資料不足時，Queue 依然會將現有的節點串接，形成的結構可能變得傾斜或不對稱，但這不影響 findMax 函數的結果。因為該函數使用遞迴搜尋法，所以會逐一走訪每個節點。 當遞迴遇到缺失的子節點時，會回傳 INT_MIN 常數作為邊界標記。由於 max()函數會自動忽略這些極小值，因此無論樹的形狀多麼不規則，程式最終還是能從所有已連接的有效節點中，找出左子樹與右子樹各自的最大值。